LAB LOG

Two SO Arms — Leader and Follower

Part A is the first arm (unchanged notes). Part B is only the files changed for the second arm.

Field	Detail
Leader (arm 1)	Mostly COM7 bring-up, later COM5 for teach / follow
Follower (arm 2 / Robo 2)	COM5 in the new files. Own limits, rest, and pose JSON
Motors	ST3215 / HS3215, six joints each
Library	python_st3215

Part A — First arm (kept)

Tests 1–4 notes are still: code first, then wiring, then the power supply was off, then IDs 1–6 read but write_goal_position failed.

FINAL tools from the first arm: disable-torque.py, rest_pos_with_s-curve.py (old rest 2048 / 759 / 3054 / 1926 / 3385 / 2301), new_save-custom_pos.py, record_motions.py, move_recorded_motion.py, collaborate_both-robots.py, hand control robot.py.

Unchanged copies are not listed again below.

Part B — Second arm only

These files were copied and then edited so the follower has its own IDs, limits, rest, and pose book. COM port in this set is COM5 unless a line says otherwise.

Test	File	What changed
B1	assign-new-ids.py	NEW — rewrite servo EEPROM IDs
B2	diagnostic-id3.py	NEW — full status dump for ID 3
B3	min-max-pos-mimic.txt	NEW — measured travel, including wrap joints
B4	middle-position.txt	NEW — hand-saved mid pose
B5	rest-position.txt + rest_pos_with_s-curve.py	New rest numbers for this arm
B6	move-robo2-to-middle-pos-new-values.py	NEW — mid pose with wrap check
B7	move to center with deceleration formula.py	Now uses per-joint mids, not 2048
B8	save_position_custom1.py	Writes recorded_poses_robo2.json
B9	move to x pose2.py	Reads recorded_poses_robo2.json
B10	move to x pose_new.py	Clamps every target to Robo 2 limits
B11	halfpos_trial.py	Same rest move, port now COM5

Test B1 — assign-new-ids.py

File: `assign-new-ids.py` **Result:** **NEW — change stored servo IDs**

What the code does

- Opens COM5 and scans IDs 0–20.
- Needs exactly six live IDs. Parks them on 200–205 first so nothing collides.
- Then writes each parked ID onto TARGET_IDS. In this file TARGET_IDS is 1–6.
- Each change: torque off, unlock EEPROM, eeprom.write_id(), lock, read the new ID back.

The comment at the top says 7–12. The lists in the file send the motors to 1–6. Use one bus with one motor if every factory ID is still 1.

What this means

Test B2 — diagnostic-id3.py

File: `diagnostic-id3.py` **Result:** **NEW — why ID 3 stalls or wraps**

What the code does

- COM5, ID 3 only. Turns torque off first.
- Prints position, load, temperature, voltage, current, EEPROM min/max, and the status byte.

Use this when a joint is hot, stuck at a stop, or the EEPROM angle window is too tight.

What this means

Test B3 — min-max-pos-mimic.txt

File: `min-max-pos-mimic.txt` **Result:** **measured travel for the follower**

Joints 3 and 5 wrap past 4095, so (min + max) / 2 is not the middle of the real arc. The file lists both the raw average and a wrap mid.

ID	Joint	Min	Max	Use this mid
1	Base	752	3443	2097
2	Shoulder	804	3209	2006
3	Elbow (wrap)	1856	28	2990
4	Wrist Pitch	843	2911	1877
5	Wrist Roll (wrap)	3752	218	4033 or 1985
6	Wrist Yaw	717	2155	1436

Two later scripts pick different mids for joint 5 (4033 vs 1985). Both are on the wrap circle. Record which one the arm actually used.

What this means

Test B4 — middle-position.txt

File: `middle-position.txt` **Result:** **hand-saved mid snapshot**

ID	Counts	Angle
1	2088	183.52°
2	2009	176.54°
3	930	81.74°
4	1870	164.36°
5	3880	341.02°
6	1431	125.77°

This mid for joint 3 (930) is the simple average, not the wrap mid 2990. Different files use different "middle" numbers.

What this means

Test B5 — rest for Robo 2

File: `rest-position.txt` and `rest_pos_with_s-curve.py` **Result:** **rest numbers changed**

What changed

- Port is COM5.
- New rest in rest-position.txt / rest_pos_with_s-curve.py:

ID	Counts	Angle
1	2085.5	183.30°
2	2014	177.06°
3	2995	263.23°
4	2024	177.89°
5	3885	341.50°
6	1429	125.60°

Robo 2 teach script uses a different rest again: 2047, 829, 2990, 2906, 2907, 713 (see B8).

What this means

Test B6 — move-robo2-to-middle-pos-new-values.py

File: `move-robo2-to-middle-pos-new-values.py` **Result:** **NEW — wrap-aware mid move**

- COM5. Targets 2097, 2006, 2990, 1877, 1985, 1436.
- in_allowed_range() treats min > max as a wrap (min…4095 and 0…max).
- Prints OK or OUT OF RANGE before the move, then the error after.

What this means

Test B7 — move to center with deceleration formula.py

File: `move to center with deceleration formula.py` **Result:** **better than all-2048**

What changed vs the first-arm version

- COM5. Each joint has its own target from min-max-pos-mimic.txt (2097, 2006, 2990, 1877, 4033, 1436).
- Still uses accel register 41 and speed 46, sync write, wait until still.
- Prints target vs actual after the move.

What this means

Test B8 — save_position_custom1.py (Robo 2)

File: `save_position_custom1.py` **Result:** **FINAL for Robo 2 teach**

What changed

- Save file is recorded_poses_robo2.json, not recorded_poses.json.
- Rest baked into this copy: 2047, 829, 2990, 2906, 2907, 713.
- Prompt is teach_robo2>. Same commands: name, l, list, q.

What this means

Test B9 — move to x pose2.py

File: `move to x pose2.py` **Result:** **FINAL play for Robo 2 JSON**

- Same as move_to_x_pos1.py, but loads recorded_poses_robo2.json on COM5.

What this means

Test B10 — move to x pose_new.py

File: `move to x pose_new.py` **Result:** **adds limit clamp + built-in MIDPOINT**

What is better than pose2

- Every target is clipped to SERVO_LIMITS before it is sent.
- Always offers a MIDPOINT pose even if the JSON is missing.

ID	Clamp min	Clamp max	Mid used
1	726	3451	2089
2	813	3205	2009
3	1	1860	931
4	846	2894	1870
5	3752	4008	3880
6	707	2156	1432

These clamps do not model wrap the same way as min-max-pos-mimic.txt. Joint 3 here is 1–1860, not 1856→28.

What this means

Test B11 — halfpos_trial.py

File: `halfpos_trial.py` **Result:** **port changed to COM5**

- Same first-arm rest dict (2048, 759, 3054…). Only the port moved. Prefer B5 / B6 for Robo 2 numbers.

What this means

Robo 2 pose book

File: `recorded_poses_robo2.json` **Result:** **follower saved poses**

Rest block: 2047, 829, 2990, 2906, 2907, 713. Named poses include STAND_UP, REST_POSITION, REST_POS_FINAL, REST. Some early saves only stored two joints.

poses.json is an older COM5 snapshot (REST_POSITION, REST1). Keep it as history, not as the live file.

Numbers that must not be mixed

Set	1	2	3	4	5	6
Arm 1 FINAL rest	2048	759	3054	1926	3385	2301
Robo 2 rest.txt	2086	2014	2995	2024	3885	1429
Robo 2 teach rest	2047	829	2990	2906	2907	713

Set	1	2	3	4	5	6
Wrap mid (mimic txt)	2097	2006	2990	1877	4033	1436
middle-position.txt	2088	2009	930	1870	3880	1431

Arm 1 and Robo 2 do not share rest or mid. Pick one table per arm and keep JSON files separate.

EEPROM ID commands (from B1)

```
servo.sram.unlock()
servo.eeprom.write_id(new_id)
new_servo = controller.wrap_servo(new_id)
new_servo.sram.lock()
new_servo.eeprom.read_id()
```

LeRobot Setup and Teleoperation
Follower Calibration
*lerobot-calibrate *
*--robot.type=so101_follower *
*--robot.port=/dev/ttyACM0 *
--robot.id=my_follower_arm
Leader Calibration
*lerobot-calibrate *
*--teleop.type=so101_leader *
*--teleop.port=/dev/ttyACM0 *
--teleop.id=my_follower_arm
Teleoperation
The leader arm was manually controlled to make the follower arm reproduce its movements.
*lerobot-teleoperate *
*--robot.type=so101_follower *
*--robot.port=/dev/ttyACM0 *
*--robot.id=my_follower_arm *
*--teleop.type=so101_leader *
*--teleop.port=/dev/ttyACM0 *
--teleop.id=my_follower_arm
Dataset Recording
A dataset was recorded by manually controlling the leader arm while the follower arm copied its movements. The task was to pick up a red cube and place it inside a box.
A negative dataset was also recorded. In this dataset, the cube was removed from the environment and the robot was not moved. This provided examples of the robot remaining stationary when the target object was absent.
ACT Policy
*Learnt about the **ACT (Action Chunking with Transformers) policy**, including the role of **weights** and **biases** in the learned model. The recorded demonstrations are used to train the policy to predict the actions required for the follower robot to perform the task.*